Experiment No: 05

Title: Implementation of Different Types of Joins with Suitable Examples

Topics to be covered:

- Inner Join
- Left Outer Join
- Right Outer Join
- Full Outer Join

SQL joins are used to combine rows from two or more tables, based on a related column between them. The most commonly used joins include:

Example Tables

Table: `employees`

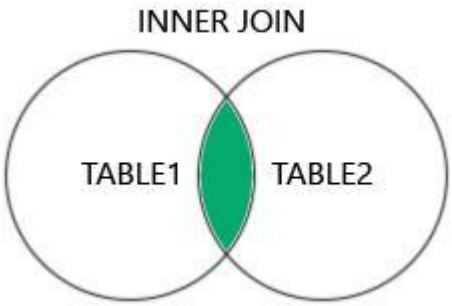
employee_id	first_name	last_name	department
1	John	Doe	IT
2	Jane	Smith	HR
3	Tom	Brown	Sales
4	Alice	Green	IT
5	Bob	White	NULL

Table: `departments`

department_id	department_name
IT	Information Tech
HR	Human Resources
Sales	Sales Department
Marketing	Marketing Dept
Admin	Administration

1. Inner Join

Description: Returns only the rows that have matching values in both tables.



Syntax Example:

```
SELECT employees.first_name, departments.department_name
FROM employees
INNER JOIN departments ON employees.department =
departments.department_id;
```

Output:

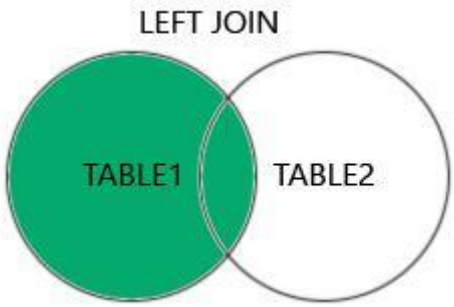
first_name	department_name
John	Information Tech
Jane	Human Resources
Tom	Sales Department
Alice	Information Tech

The INNER JOIN syntax can be rewritten using a more traditional approach or a different aliasing style, and both will yield the same output.

```
SELECT e.first_name, d.department_name
FROM employees e
INNER JOIN departments d
ON e.department = d.department_id;
```

2. Left Outer Join (LEFT JOIN)

Description: Returns all rows from the left table and the matched rows from the right table. If no match, NULL values appear for the right table.



Syntax Example:

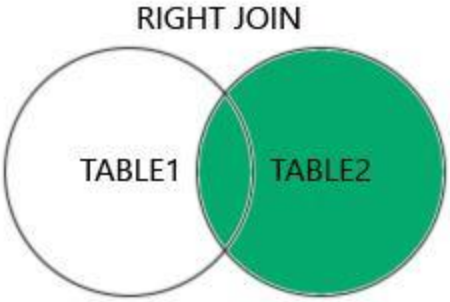
```
SELECT employees.first_name, departments.department_name
FROM employees
LEFT JOIN departments ON employees.department =
departments.department_id;
```

Output:

first_name	department_name
John	Information Tech
Jane	Human Resources
Tom	Sales Department
Alice	Information Tech
Bob	NULL

3. Right Outer Join (RIGHT JOIN)

Description: Returns all rows from the right table and the matched rows from the left table. If no match, NULL values appear for the left table.



Syntax Example:

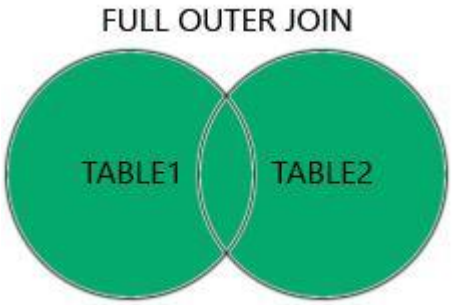
```
SELECT employees.first_name, departments.department_name
FROM employees
RIGHT JOIN departments ON employees.department =
departments.department_id;
```

Output:

first_name	department_name
John	Information Tech
Alice	Information Tech
Jane	Human Resources
Tom	Sales Department
NULL	Marketing Dept
NULL	Administration

4. Full Outer Join

Description: Returns all records when there is a match in either left or right table. If there's no match, NULLs are shown.



Syntax Example (for MySQL Workaround):

```
SELECT employees.first_name, departments.department_name
FROM employees
LEFT JOIN departments ON employees.department =
departments.department_id

UNION

SELECT employees.first_name, departments.department_name
FROM employees
RIGHT JOIN departments ON employees.department =
departments.department_id;
```

Output:

first_name	department_name
John	Information Tech
Alice	Information Tech
Jane	Human Resources
Tom	Sales Department
Bob	NULL
NULL	Marketing Dept
NULL	Administration

Example Questions and Solutions

Q1: Display all employees with their department names using INNER JOIN.

```
SELECT e.first_name, d.department_name
FROM employees e
INNER JOIN departments d ON e.department = d.department_id;
```

Q2: Display all employees and their department names using LEFT JOIN.

```
SELECT e.first_name, d.department_name
```

```
FROM employees e
LEFT JOIN departments d ON e.department = d.department_id;
```

Q3: Display all departments and associated employee names using RIGHT JOIN.

```
SELECT e.first_name, d.department_name
FROM employees e
RIGHT JOIN departments d ON e.department = d.department_id;
```

Q4: Display all employees and departments even if there's no match using FULL OUTER JOIN.

```
SELECT e.first_name, d.department_name
FROM employees e
LEFT JOIN departments d ON e.department = d.department_id

UNION

SELECT e.first_name, d.department_name
FROM employees e
RIGHT JOIN departments d ON e.department = d.department_id;
```

SQL Self JOIN:

https://www.programiz.com/sql/self-join

SQL CROSS JOIN:

https://www.programiz.com/sql/cross-join

1. Self Join in MySQL

A **self join** is a regular join but the table is joined with itself.

Example:

```
SELECT A.id, A.name AS Employee, B.name AS Manager
FROM employees A
JOIN employees B ON A.manager_id = B.id;
```

- In this example, the `employees` table is joined with itself to match each employee with their manager.
- Aliases (`A`, `B`) are used to differentiate the two instances of the same table.

2. Cross Join in MySQL

A **cross join** returns the Cartesian product of two tables — every row from the first table combined with every row from the second.

Example:

```
SELECT *
FROM products
CROSS JOIN categories;
```

- If `products` has 5 rows and `categories` has 3 rows, the result will have **15 rows** (5 × 3).
- Cross join is useful in scenarios like generating combinations.

The JOIN operation

https://sqlzoo.net/wiki/The_JOIN_operation

More JOIN operations

https://sqlzoo.net/wiki/More_JOIN_operations

Lab Task-1: Sample Tables

Table: Departments

```
CREATE TABLE Departments (
    dept_id INT,
    dept_name VARCHAR(30)
);
```

dept_id	dept_name
1	IT
2	HR
3	Sales
4	Marketing
5	Finance

Table: Employees

```
CREATE TABLE Employees (
    emp_id INT,
    emp_name VARCHAR(30),
    dept_id INT
);
```

emp_id	emp_name	dept_id
101	John	1
102	Jane	2
103	Alice	3
104	Bob	NULL
105	Tom	1
106	Lisa	6

1. Retrieve all employees who are assigned to a valid department using INNER JOIN.
2. List all employees along with their department name (if any) using LEFT JOIN.
3. Show all departments along with employees working in them, using RIGHT JOIN.
4. Display all employees and all departments, showing NULL where no match exists.
5. List employees who **do not belong to any department** using LEFT JOIN and WHERE dept_name IS NULL.
6. List departments that **do not have any employees assigned** using RIGHT JOIN and WHERE emp_name IS NULL.
7. Use table aliases (E and D) to list employee names and department names using INNER JOIN.
8. List employee names with department names using LEFT JOIN, sorted by department name.
9. Show employees working in the 'IT' department using INNER JOIN and WHERE.
10. Identify employees whose dept_id is not listed in the Departments table.

    Use: LEFT JOIN with WHERE D.dept_id IS NULL

Note: *MySQL does not support FULL OUTER JOIN directly; use UNION of LEFT and RIGHT JOIN.*

For **FULL OUTER JOIN in MySQL**, use this pattern:

```
SELECT E.emp_id, E.emp_name, D.dept_name
FROM Employees E
LEFT JOIN Departments D ON E.dept_id = D.dept_id
UNION
SELECT E.emp_id, E.emp_name, D.dept_name
FROM Employees E
RIGHT JOIN Departments D ON E.dept_id = D.dept_id;
```

Lab Task-2: Sample Tables

Table: Courses

```
CREATE TABLE Courses (
    course_id INT,
    course_name VARCHAR(40)
);
```

Table: Students

```
CREATE TABLE Students (
    student_id INT,
    student_name VARCHAR(40),
    course_id INT
);
```

Sample Data

Courses

course_id	course_name
201	Database
202	Networking
203	Programming
204	AI
205	Cyber Security

Students

student_id	student_name	course_id
301	Rahim	201
302	Karim	202
303	Hasan	203
304	Rafiq	NULL
305	Tania	201
306	Sima	210

Questions

1. Retrieve all students who are enrolled in valid courses using **INNER JOIN**.
2. List all students along with their course name (if any) using **LEFT JOIN**.
3. Show all courses along with the students enrolled in them using **RIGHT JOIN**.
4. Display all students and all courses, showing **NULL** where no match exists (**FULL OUTER JOIN**).
5. Find students who are **not enrolled in any course** using **LEFT JOIN** and `WHERE course_name IS NULL`.
6. Find courses that currently **have no students enrolled** using **RIGHT JOIN** and `WHERE student_name IS NULL`.
7. Use table aliases (`S` and `C`) to list student names and course names using **INNER JOIN**.
8. List student names with their course names using **LEFT JOIN**, sorted by course name.
9. Show students enrolled in the **Database** course using **INNER JOIN** and `WHERE`.
10. Identify students whose `course_id` is **not listed** in the Courses table. Use: **LEFT JOIN** with `WHERE C.course_id IS NULL`.

Lab Task-3: Use the Lab Task-2 tables

Questions

1. Display the **number of students enrolled** in each course using `INNER JOIN` and `GROUP BY`.
2. List all students along with their courses, but highlight those whose `course_id` **does not exist** in the Courses table.
3. Find all courses where **more than one student** is enrolled.
4. Retrieve the **names of students who are not enrolled in any course** (use `LEFT JOIN` + `WHERE`).
5. Show all courses that **do not have any students enrolled** (use `RIGHT JOIN` + `WHERE`).
6. Display a list of students and their courses, but replace `NULL` with `"Not Assigned"` using `COALESCE()`.
7. Find the **course(s) with the maximum number of enrolled students**.
8. List students whose names **start with 'R'** along with their course names.
9. Show students along with course names, but only for those courses whose name **contains the word 'ing'**.
10. Display all students and their courses, but if a student's `course_id` is invalid (not in Courses table), show `"Invalid Course"` instead.